SwissGov-RSD with Apertus : Short Method Description

Track 2A (UZH) · HackApertus 2026

Task: Token-level cross-lingual semantic-difference recognition (EN–DE, EN–FR, EN–IT)

Model: swiss-ai/Apertus-v1.5-8B

Metric: macro-averaged token-level Spearman correlation

1. Overview

For a pair of related documents in two languages, we score every whitespace token by how strongly its meaning diverges from the aligned content in the other document, capturing symmetric differences, additions, and omissions. Prior work shows prompted LLMs perform poorly here and lose to unsupervised encoder-embedding methods. We therefore do not prompt Apertus to emit scores. We use Apertus purely as an encoder and compute alignment geometrically in its hidden-state space an unsupervised, training-free method we call **DiffAlign-Apertus**. Two choices drive the result: scoring from an intermediate layer rather than the output layer, and mean-centering token embeddings before comparison.

2. Method

Encoding. Each document is passed once through Apertus-v1.5-8B with `output_hidden_states=True`; we take the per-token hidden states of a single layer ℓ. (Apertus-v1.5 is a multimodal/Omni checkpoint; we extract its inner language-model module to obtain text hidden states.)

Alignment score. Let H_A and H_B be the subword hidden states of documents A and B at layer ℓ. For each token a_i in A, its difference score is one minus its best cosine alignment to any token in B:

$$\mathrm{diff}(a_i) = 1 - \max_j \cos\big(h(a_i), h(b_j)\big)$$

The score for B is computed symmetrically against A. A token with no well-aligned counterpart (addition, omission, mistranslation) receives a high difference score. The method is unsupervised: no labels, no training, no test-set leakage.

Mean-centering. Apertus's embedding space is anisotropic — a shared language-identity offset dominates the geometry and weakens raw cosine retrieval. We subtract the per-document mean token vector before scoring:

$$\tilde{h}(a_i) = h(a_i) - \frac{1}{n}\sum_k h(a_k)$$

This exposes the semantic signal and is the single largest lever in our system.

Layer selection. Cross-lingual alignment in decoder LLMs peaks in the middle of the network (≈60–80% depth), where representations are most language-agnostic, before diverging in the final layers for next-token prediction. A dev-set sweep confirms this on Apertus, with the peak at layer 20 (62.5% depth).

Subword→word pooling. Gold labels are over whitespace words; Apertus uses subword tokens. Using the tokenizer's character offset mapping, we assign each subword to its whitespace word and pool by max (any strongly-diverging piece marks the word as different). This step is essential a subword/word mismatch silently destroys the correlation. Punctuation is excluded to match the official evaluator.

3. Experimental setup

- **Data:** ZurichNLP/SwissGov-RSD, development split (168 docs/language, 504 total). The held-out test split is never used in development gated behind an explicit flag, run once at the end with the dev-frozen configuration.
- **Scoring:** official `evaluate_predictions_admin.py` (defines preprocessing and punctuation handling); we report Spearman and Kendall τ-b.
- **Hyperparameters** (layer, centering) selected on dev only.
- **Hardware:** NVIDIA H100 (80 GB, Apertus in bfloat16.

4. Results (development split, official evaluator)

Layer sweep and centering ablation (macro-averaged token-level Spearman):

Configuration	Layer	Depth	Center	DE	FR	IT	Macro
Output, uncentered	31	100%	✗	0.073	0.060	0.100	0.078
Output, centered	31	100%	✓	0.172	0.112	0.179	0.154
Upper, centered	28	87.5%	✓	0.229	0.145	0.242	0.205
Mid-upper, centered	24	75.0%	✓	0.260	0.163	0.275	0.233
Mid, centered	16	50.0%	✓	0.282	0.154	0.291	0.242
Peak, centered (ours)	**20**	**62.5%**	✓	**0.288**	**0.173**	**0.305**	**0.255**

Findings. (1) Mean-centering roughly doubles alignment at the output layer (0.078 → 0.154 macro): Apertus's limitation is anisotropy, not missing meaning. (2) Alignment peaks at intermediate depth layer 20 is best at 0.255 macro, rising toward the middle and falling at both ends.

Context. Published SwissGov-RSD results report prompted LLMs at ~0.02–0.04 and the best unsupervised DiffAlign (XLM-R/Qwen3 encoders) at ~0.155 macro Spearman. Our dev result of 0.255 is

substantially above the prompting baselines and the published unsupervised figures. *Caveat:* 0.255 is a dev-set number; the published figures are on the test split, so this is an indicative, not like-for-like, comparison. The final test-set figure (single run, frozen config) is reported in the technical report.

5. Efficiency (secondary criterion)

DiffAlign-Apertus is a single forward pass with no autoregressive decoding. On one H100 it processes the 504-document dev set in ≈75 s (~0.15 s per document pair) with ≈16.5 GB peak VRAM. By construction it emits exactly one score per input token, so it has no JSON-parsing or token-count failures the main failure mode of prompting on long documents. A well-tuned 8B configuration is thus both accurate and cheap.

6. Reproducibility

Code in `src/` (model loader, DiffAlign, subword pooling, official-format export) with conda/Slurm scripts for clusters.

Final configuration: `--method diffalign --layer 20 --center on` on Apertus-v1.5-8B. A few-shot prompting run on the same model records the required before/after baseline.

Repo: github.com/wassim-mezghanni/Hack-Apertus-Swiss-open-source-AI-hackathon-series

Summary. Treating Apertus as an encoder, correcting its anisotropic geometry via mean-centering, and reading from the layer where cross-lingual alignment peaks turns an 8B decoder weak under prompting into a competitive, efficient, fully unsupervised semantic-difference detector.